

UD4 - Acceso a datos con PHP

Alumno: Francisco Pérez Ruiz **Asignatura:** Desarrollo Web en Entorno Servidor (DWES)

Introducción

En esta unidad damos el paso de "hacer cosas con PHP" a "hacer cosas con PHP **conectado a una base de datos**". Es uno de los apartados más prácticos, porque casi cualquier aplicación web real necesita guardar y leer datos en algún sitio.

Durante la unidad se ven dos formas de conectarse a MySQL desde PHP:

- **MySQLi:** la extensión "clásica", específica de MySQL. Tiene una API procedural y otra orientada a objetos.
- **PDO** (PHP Data Objects): una capa de abstracción que sirve para muchas bases de datos (MySQL, SQLite, PostgreSQL, etc.). Es la opción más moderna.

También se tocan temas alrededor: transacciones, prevención de **inyección SQL**, patrón **MVC**, **SQLite** como alternativa ligera y, al final, manejo de **ficheros** y generación de **PDFs** con FPDF.

Un poco de historia de PHP

PHP nació en 1994 de la mano de Rasmus Lerdorf. Empezó como una herramienta personal para gestionar su página web, pero con el tiempo se convirtió en uno de los lenguajes más usados en el mundo del desarrollo web.

Algunos hitos importantes:

- **PHP 3 (1998):** se sentaron las bases del lenguaje moderno: analizador sintáctico nuevo, soporte para muchas bases de datos.
- **PHP 4 (2000):** apareció el motor Zend, que aumentó mucho la velocidad de ejecución.

- **PHP 5 (2004):** la POO en PHP se vuelve seria de verdad. Estuvo en el mercado más de 11 años.
 - **PHP 7 (2015):** mejoras enormes de rendimiento (la versión 6 se saltó). De aquí se pasa a PHP 8.
 - **PHP 8 (2020 en adelante):** JIT, tipos más estrictos, atributos.
-

P1 - Primera conexión con MySQLi

El punto de partida: conectarse a una base de datos `dwes` en MySQL y leer una tabla `ud4_persona`.

Pasos clave:

1. Crear la base de datos `dwes` desde phpMyAdmin.
2. Crear la tabla `ud4_persona` con los campos `id`, `nombre`, `apellidos`, `telefono`.
3. Desde PHP, usar `mysqli_connect("localhost", "root", "", "dwes")` para conectarse.
4. Lanzar una `SELECT` con `mysqli_query()` y recorrer el resultado con `foreach`.

Código de la conexión + consulta:

```
1 <?php
2 // 1. CONEXIÓN con La BBDD:
3 // "SERVIDOR", "USUARIO", "CONTRASEÑA", "BASE DE DATOS"
4
5
6 try {
7     //https://www.php.net/manual/es/function.mysqli-connect.php
8     $conexion = mysqli_connect("localhost", "root", "", "dwes");
9 } catch (Exception $e) {
10     echo "<br> 1. Se ha producido el siguiente error: " . $e->getMessage();
11     echo "<br> 2. Falló la conexión: " . mysqli_connect_error();
12     exit();
13 }
14
15
16 // 3. CONSULTA A LA BASE DE DATOS
17 echo "<h3>Listado de usuarios</h3>";
18 $consulta = "SELECT * FROM `ud4_persona`";
19 $listaUsuarios = mysqli_query($conexion, $consulta);
20 print_r($listaUsuarios);
21 echo "<br>";
22 // 4. COMPROBAMOS SI EL SERVIDOR NOS HA DEVUELTO RESULTADOS
23 if ($listaUsuarios) {
24
25     // RECORREMOS CADA RESULTADO QUE NOS DEVUELVE EL SERVIDOR
26     foreach ($listaUsuarios as $usuario) {
27         echo "
28             $usuario[id]
29             $usuario[nombre]
30             $usuario[apellidos]
31             $usuario[telefono]
32             <br>
33         ";
34     }
35     mysqli_close($conexion);
36 }
37
```

P2 - CRUD completo con MySQLi y transacciones

Este apartado da un salto en complejidad. Ya no es solo "leer datos", sino hacer **CRUD completo** (Create, Read, Update, Delete) con buenas prácticas: **prepared**

statements (para evitar inyección SQL) y **transacciones** (para garantizar que las operaciones críticas se hagan en bloque o se reviertan).

Conexión centralizada

En vez de copiar la conexión en cada archivo, se centraliza en una función `db()` que devuelve siempre la misma conexión.

conexion_mysql.php · líneas 1-32

```
1  <?php
2  /*  $cn = mysqli_connect("localhost", "root", "", "dwes");
3      if (!$cn) { die("Error de conexión: " . mysqli_connect_error()); }
4      mysqli_set_charset($cn, "utf8mb4");
5
6
7      mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
8  */
9  function db(): mysqli
10 {
11     static $cn = null;
12     if ($cn instanceof mysqli) {
13         return $cn;
14     }
15     $host = "localhost";
16     $user = "root";
17     $pass = "";
18     $db = "dwes";           // <-- Base de datos: dwes
19
20
21     $cn = mysqli_connect($host, $user, $pass, $db);
22     mysqli_set_charset($cn, "utf8mb4");
23     echo "Conexión OK <br>";
24     return $cn;
25 }
26
27
28 // mysqli_close($cn);
29
30
31 ?>
32 <?php
```

Insertar con transacción

`mysqli_begin_transaction` + `mysqli_commit` (o `mysqli_rollback` si algo falla). Si una de las dos inserciones falla, **ninguna** se guarda.

```
1 <?php
2 /* ===== insertar_multiples_tx.php =====
3     INSERT atómico de 2 filas en dwes.ud4_empresa (o ninguna si falla).
4 */
5 require_once __DIR__ . '/conexion_mysqli.php';
6 $cn = db();
7 //echo "<br>Conexión establecida.<br>";
8
9
10 // bloque try-catch-finally para manejar la transacción
11 try {
12     // Inicio de la transacción
13     mysqli_begin_transaction($cn);
14
15
16     // Preparar la sentencia INSERT con marcadores de posición: ?, ?, ?
17     $stmt = mysqli_prepare($cn,
18         "INSERT INTO ud4_empresa (nombre, cargo, salario) VALUES (?, ?, ?)"
19     );
20
21
22     // 1ª fila
23     $nombre = "Alberto"; $cargo = "Interfaces"; $salario = 20000.00;
24
25
26     // Vincular los parámetros y ejecutar la sentencia
27     mysqli_stmt_bind_param($stmt, "ssd", $nombre, $cargo, $salario);
28
29
30     // Ejecutar el INSERT
31     mysqli_stmt_execute($stmt);
32
33
34     // 2ª fila
35     $nombre = "Maria"; $cargo = "Desarrollo"; $salario = 30000.00;
36     mysqli_stmt_bind_param($stmt, "ssd", $nombre, $cargo, $salario);
37     mysqli_stmt_execute($stmt);
38
39
40     mysqli_stmt_close($stmt);
41     mysqli_commit($cn);
42     echo "Insertadas 2 filas en dwes.ud4_empresa.";
43 } catch (Throwable $e) {
44     mysqli_rollback($cn);
45     echo "Insert cancelado: " . $e->getMessage();
46 } finally {
47     mysqli_close($cn);
48 }
49
```

Select con bloqueo **FOR UPDATE**

Lectura de una fila con bloqueo, para garantizar que nadie más la modifique entre que la leo y la actualizo.

```

1 <?php
2 /* ===== select_for_update_tx.php =====
3     Lectura con bloqueo de fila (SELECT ... FOR UPDATE) + actualización consistente.
4 */
5 require_once __DIR__ . '/conexion_mysql.php';
6 $cn = db();
7
8
9 //selecciona el id para encontrar y actualizar
10 $id = 2;
11
12
13 try {
14     mysqli_begin_transaction($cn);
15
16
17     // 1) Leer con bloqueo
18     $stmt = mysqli_prepare($cn,
19         "SELECT salario FROM ud4_empresa WHERE id = ? FOR UPDATE"
20     );
21     mysqli_stmt_bind_param($stmt, "i", $id);
22     mysqli_stmt_execute($stmt);
23     $res = mysqli_stmt_get_result($stmt);
24     $row = mysqli_fetch_assoc($res);
25     if (!$row) {
26         throw new RuntimeException("Empleado no existe (id=$id).");
27     }
28
29
30     echo "Hemos leído el salario con FOR UPDATE.<br>";
31     echo "Salario actual: " . $row['salario'] . "<br>";
32     // Obtener salario actual
33     $salarioActual = (float)$row['salario'];
34     mysqli_free_result($res);
35     mysqli_stmt_close($stmt);
36
37
38     // 2) Calcular nuevo salario (ej. +100€)
39     $nuevo = $salarioActual ;
40
41
42     // 3) Actualizar
43     $stmt = mysqli_prepare($cn,
44         "UPDATE ud4_empresa SET salario = ? WHERE id = ?"
45     );
46     mysqli_stmt_bind_param($stmt, "di", $nuevo, $id);
47     mysqli_stmt_execute($stmt);
48     mysqli_stmt_close($stmt);
49
50
51     mysqli_commit($cn);
52     echo "Salario actualizado con FOR UPDATE en dwes.ud4_empresa.";
53 }

```



```

53 } catch (Throwable $e) {
54     mysqli_rollback($cn);
55     echo "Error transaccional: " . $e->getMessage();
56 } finally {
57     mysqli_close($cn);
58 }
59

```

Actualizar con validación

Update protegido: si no se afecta exactamente 1 fila, se lanza una excepción y se hace **rollback**.

actualizar.php · líneas 1-36

```

1  <?php
2  mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);
3  $cn = mysqli_connect("localhost","root","","dwes");
4  mysqli_set_charset($cn, "utf8mb4");
5
6
7  $id = 1;                // empleado objetivo
8  $nuevo = 2000.00;
9
10
11 try {
12     mysqli_begin_transaction($cn);
13
14
15     $stmt = mysqli_prepare($cn,
16         "UPDATE ud4_empresa SET salario = ? WHERE id = ?"
17     );
18     mysqli_stmt_bind_param($stmt, "di", $nuevo, $id);
19     mysqli_stmt_execute($stmt);
20
21
22     if (mysqli_stmt_affected_rows($stmt) !== 1) {
23         throw new RuntimeException("No se actualizó exactamente 1 fila.");
24     }
25
26
27     mysqli_stmt_close($stmt);
28     mysqli_commit($cn);
29     echo "✅ Salario actualizado.";
30 } catch (Throwable $e) {
31     mysqli_rollback($cn);
32     echo "❌ Update revertido: " . $e->getMessage();
33 } finally {
34     mysqli_close($cn);
35 }
36

```

Borrar con validación

Misma idea para el **DELETE**. Si no se borra 1 fila, se cancela.

delete.php · líneas 1-37

```
1 <?php
2 /* ===== borrar_empleado_tx.php =====
3     DELETE seguro bajo transacción (valida filas afectadas = 1).
4 */
5 require_once __DIR__ . '/conexion_mysqli.php';
6 $cn = db();
7
8
9 // Empleado a borrar
10 $id = 1;
11
12
13 try {
14     mysqli_begin_transaction($cn);
15
16
17     $stmt = mysqli_prepare($cn, "DELETE FROM ud4_empresa WHERE id = ?" );
18     mysqli_stmt_bind_param($stmt, "i", $id);
19     mysqli_stmt_execute($stmt);
20
21
22     if (mysqli_stmt_affected_rows($stmt) !== 1) {
23         throw new RuntimeException("No se eliminó exactamente 1 fila (¿ID inexistente?).");
24     }
25
26
27     mysqli_stmt_close($stmt);
28     mysqli_commit($cn);
29     echo "Empleado eliminado de dwes.ud4_empresa.";
30 } catch (Throwable $e) {
31     mysqli_rollback($cn);
32     echo "Delete revertido: " . $e->getMessage();
33 } finally {
34     mysqli_close($cn);
35 }
36
37 ?>
```

Inyección SQL (mal ejemplo)

Aquí se ve por qué **nunca** se debe concatenar la entrada del usuario directamente en una consulta. Con ' **OR '1'='1** se consigue que se muestren todos los empleados.

```
1 <?php
2 // conexion_mysqli.php debe proporcionar db()
3 require_once 'conexion_mysqli.php';
4 $cn = db();
5
6
7 // Supongamos un formulario que envía $_GET['nombre']
8 //$nombre = $_GET['nombre'] ?? '';
9 $nombre= "' OR '1'='1";
10 //$nombre= "Laura";
11
12
13 // Vulnerable: concatenación directa de usuario en la consulta
14 $sql = "SELECT id, nombre, cargo, salario FROM ud4_empresa WHERE nombre = '$nombre'";
15
16
17 $res = mysqli_query($cn, $sql);
18 while ($fila = mysqli_fetch_assoc($res)) {
19     echo htmlspecialchars($fila['id']) . ' - ' . htmlspecialchars($fila['nombre']) . "<br>";
20 }
21 echo "<br> Se han mostrado todos los empleados si la inyección tuvo éxito.<br>";
22
23
24 // Cierre de conexión
25 mysqli_close($cn);
26 ?>
27
```

La forma correcta es usar marcadores (? o :nombre) y bind_param / bindValue, como en los ejemplos anteriores.

P3 - Conexión con PDO

PDO es la alternativa "moderna" a MySQLi. La gran ventaja: el mismo código sirve para MySQL, SQLite, PostgreSQL, etc., cambiando solo el DSN.

Conexión centralizada con PDO

```

1 <?php
2 /* conexión centralizada con PDO (MySQL/MariaDB) */
3 function pdo(): PDO {
4     static $pdo = null;
5     if ($pdo instanceof PDO) return $pdo;
6
7
8     $host = 'localhost';
9     $db   = 'dwes';
10    $user = 'root';
11    $pass = '';
12    $charset = 'utf8mb4';
13
14
15    $dsn = "mysql:host=$host;dbname=$db;charset=$charset";
16    $opts = [
17        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION, // excepciones
18        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,        // arrays asociativos
19        PDO::ATTR_EMULATE_PREPARES   => false,                    // prepares nativos
20    ];
21    $pdo = new PDO($dsn, $user, $pass, $opts);
22    return $pdo;
23 }
24 ?>

```

Detalle importante: configuro `ERRMODE_EXCEPTION` para que cualquier error de SQL lance una excepción (mucho más fácil de manejar que comprobar códigos a mano), y `EMULATE_PREPARES => false` para usar prepares nativos del motor.

Listado de canciones con CSS

Ejemplo práctico: conexión PDO + lectura completa de la tabla `ud4_musica` + presentación en HTML con la librería Pico.css.

Primera parte (conexión + consulta):

```
1 <?php
2 /* leer_musica.php - Listado simple con PDO (MySQL/MariaDB) */
3
4
5 $host = 'localhost';
6 $db   = 'dwes';
7 $user = 'root';
8 $pass = '';
9 $charset = 'utf8mb4';
10
11
12 $dsn = "mysql:host=$host;dbname=$db;charset=$charset";
13
14
15 try {
16     // Conexión PDO
17     $pdo = new PDO($dsn, $user, $pass, [
18         PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
19         PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
20         PDO::ATTR_EMULATE_PREPARES   => false,
21     ]);
22
23
24     // Consulta (lee toda la tabla)
25     $sql = "SELECT id, title, artist, genre, release_year, duration, unit_price, quantity_in_stock
26           FROM ud4_musica
27           ORDER BY id DESC";
28     $stmt = $pdo->query($sql);
29     $rows = $stmt->fetchAll();
30
31
32 } catch (Throwable $e) {
33     http_response_code(500);
34     die("Error: " . $e->getMessage());
35 }
36 ?>
```

Segunda parte (HTML con la tabla):

```

37 <!doctype html>
38 <html lang="es">
39 <meta charset="utf-8">
40 <title>Listado ud4_musica</title>
41 <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@picocss/pico@1/css/pico.min.css"/>
42 <body>
43 <h1>Listado de canciones (ud4_musica)</h1>
44 <table border="1" cellpadding="6">
45   <tr>
46     <th>ID</th><th>Title</th><th>Artist</th><th>Genre</th>
47     <th>Year</th><th>Duration</th><th>Price</th><th>Stock</th>
48   </tr>
49   <?php foreach ($rows as $r): ?>
50     <tr>
51       <td><?= htmlspecialchars($r['id']) ?></td>
52       <td><?= htmlspecialchars($r['title']) ?></td>
53       <td><?= htmlspecialchars($r['artist']) ?></td>
54       <td><?= htmlspecialchars($r['genre'] ?? '') ?></td>
55       <td><?= htmlspecialchars($r['release_year'] ?? '') ?></td>
56       <td><?= htmlspecialchars($r['duration'] ?? '') ?></td>
57       <td><?= number_format((float)$r['unit_price'], 2) ?></td>
58       <td><?= htmlspecialchars($r['quantity_in_stock']) ?></td>
59     </tr>
60   <?php endforeach; ?>
61 </table>
62 </body>
63 </html>

```

P5_Login - Login con patrón MVC

Aquí se introduce el **patrón Modelo-Vista-Controlador (MVC)** aplicado a un sistema de login. La idea es separar bien las responsabilidades:

- **Modelo** (*Persona.php*): lógica de negocio, acceso a datos.
- **Vista** (*Login.php*, *welcome.php*): solo HTML.
- **Controlador** (*PersonaController.php*): recibe la petición, llama al modelo y elige la vista.
- **Configuración** (*config/database.php*): conexión PDO encapsulada en una clase *Database*.

Conexión *database.php*

```
1 <?php
2 class Database {
3     private $host = 'localhost';
4     private $dbName = 'dwes';
5     private $username = 'root';
6     private $password = '';
7     private $conn;
8
9     public function connect() {
10         try {
11
12             $this->conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
13             return $this->conn;
14         } catch (PDOException $e) {
15             die("Error de conexión: " . $e->getMessage());
16         }
17     }
18 }
19 ?>
```

Modelo Persona.php

```
1 <?php
2 require_once __DIR__ . '/../../config/database.php';
3
4 class Persona {
5     private $conn;
6
7     /**
8      * The function creates a new Database object and establishes a connection to the database.
9      */
10    public function __construct() {
11        $db = new Database();
12        $this->conn = $db->connect();
13    }
14
15    /**
16     * The Login function queries a database table to retrieve a record based on the provided name and
17     * phone number.
18     *
19     * @param usuario The 'usuario' parameter in the 'login'
20     * @param contrasena The parameter 'contrasena' is a string that represents the password of the user trying to log in.
21     * @return The 'login' function is returning a single record from the 'persona'
22     */
23    public function login($usuario, $contrasena) {
24        try {
25            $query = "SELECT * FROM ud4_persona WHERE usuario = :usuario AND contrasena = :contrasena";
26            $stmt = $this->conn->prepare($query);
27            $stmt->bindParam(':usuario', );
28            $stmt->bindParam(':contrasena', $contrasena);
29            $stmt->execute();
30            return $stmt->fetch(PDO::FETCH_ASSOC); // Devuelve un registro si lo encuentra
31        } catch (PDOException $e) {
32            die("Error al realizar la consulta: " . $e->getMessage());
33        }
34    }
35 }
36 ?>
```

Controlador **PersonaController.php**


```

1 <?php
2 require_once __DIR__ . '/../models/Persona.php';
3
4 class PersonaController {
5     public function gestionLogin() {
6         if ($_SERVER['REQUEST_METHOD'] === 'POST') {
7             $usuario = $_POST['usuario'] ?? '';
8             $contrasena = $_POST['contrasena'] ?? '';
9
10            $personaModel = new Persona();
11
12            $persona = $personaModel->login($usuario, $contrasena);
13
14            if ($persona) {
15                // Redirige al usuario si las credenciales son correctas
16                header('Location: welcome.php');
17                echo "Autenticación exitosa";
18                exit();
19            } else {
20                // En caso de error, muestra un mensaje
21                $error = "Credenciales incorrectas.";
22                require_once __DIR__ . '/../views/login.php';
23            }
24        } else {
25            require_once __DIR__ . '/../views/login.php';
26        }
27    }
28 }
29 ?>

```

Punto de entrada `index.php`

```

1 <?php
2 require_once __DIR__ . '/app/controllers/PersonaController.php';
3
4 $controller =
5 $controller->handleLogin();
6 ?>

```

P5_CRUD_MVC - CRUD completo de coches con MVC

Misma arquitectura MVC pero ahora con todas las operaciones CRUD aplicadas a una tabla **coches**. El controlador tiene los métodos típicos:

- **index()** → muestra todos los coches.
- **crear()** → inserta uno nuevo.
- **editar()** → modifica uno existente.
- **eliminar()** → borra uno.

Modelo **Coche.php**

Coche.php · líneas 1-35

```
1 <?php
2 class Coche {
3     private $pdo;
4
5     public function __construct($pdo) {
6         $this->pdo = $pdo;
7     }
8
9     public function getAll() {
10         $stmt = $this->pdo->query("SELECT * FROM ud4_coches");
11         return $stmt->fetchAll(PDO::FETCH_ASSOC);
12         //Viene en el Punto 3 PDO
13     }
14
15     public function crear($marca, $modelo, $anio) {
16         $stmt = $this->pdo->FALTACODIGO("INSERT INTO ud4_coches(marca, modelo, anio) VALUES (:marca, :modelo, :anio)");
17         $stmt->execute([':marca' => $marca, ':modelo' => $modelo, ':anio' => $anio]);
18     }
19
20     public function editar($id, $marca, $modelo, $anio) {
21         $stmt = $this->pdo->prepare("UPDATE ud4_coches SET marca = :marca, modelo = :modelo, anio = :anio WHERE id = :id");
22         $stmt->execute([':marca' => $marca, ':modelo' => $modelo, ':anio' => $anio, ':id' => $id]);
23     }
24
25     public function eliminar($id) {
26         $stmt = $this->pdo->prepare("DELETE FROM ud4_coches WHERE id = :id");
27         $stmt->execute([':id' => $id]);
28     }
29
30     public function getById($id) {
31         $stmt = $this->pdo->prepare("SELECT * FROM ud4_coches WHERE id = :id");
32         $stmt->execute([':id' => $id]);
33         return $stmt->fetch(PDO::FETCH_ASSOC);
34     }
35 }
```

Controlador **CocheController.php**

```
1 <?php
2 include 'db.php';
3 include 'models/Coche.php';
4
5 class CocheController {
6     private $cocheModel;
7
8     public function __construct($pdo) {
9         $this->cocheModel = new Coche($pdo);
10    }
11
12    public function index() {
13        $coches = $this->cocheModel->getAll();
14        include 'views/coches/index.php';
15    }
16
17    public function crear() {
18        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
19            $this->cocheModel->crear($_POST['marca'], $_POST['modelo'], $_POST['anio']);
20            header("Location: index.php");
21        }
22    }
23
24    public function editar() {
25        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
26            $this->cocheModel->editar($_POST['id'], $_POST['marca'], $_POST['modelo'], $_POST['anio']);
27            header("Location: index.php");
28        }
29    }
30
31    public function eliminar() {
32        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
33            $this->cocheModel->eliminar($_POST['id']);
34            header("Location: index.php");
35        }
36    }
37 }
38 ?>
```

Punto de entrada **index.php**

index.php · líneas 1-22

```
1 <?php
2 include 'app/controllers/CocheController.php';
3
4 $controller = new CocheController($pdo);
5
6 $action = $_GET['action'] ?? 'index';
7
8 switch ($action) {
9     case 'crear':
10         $controller->crear();
11         break;
12     case 'editar':
13         $controller->editar();
14         break;
15     case 'eliminar':
16         $controller->eliminar();
17         break;
18     default:
19         $controller->index();
20         break;
21 }
22 ?>
```

P6 - CRUD con SQLite

En este apartado se cambia el motor: en vez de MySQL/MariaDB se usa **SQLite**. SQLite no necesita un servidor: la base de datos es **un archivo dentro del propio proyecto**.

Esto es muy práctico para proyectos pequeños o pruebas, porque no hay que tener nada arrancado (ni el puerto 3306 de MySQL).

config.php (crea la BD y la tabla si no existen, e inserta jugadores de prueba)

```
1 <?php
2 declare(strict_types=1);
3
4 // Ruta al fichero SQLite
5 $dbDir = __DIR__ . '/data';
6 $dbFile = $dbDir . '/jugadores.sqlite';
7
8 if (!is_dir($dbDir)) {
9     mkdir($dbDir, 0777, true);
10 }
11
12 $pdo = new PDO('sqlite:' . $dbFile);
13 $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
14
15 // Crear tabla si no existe
16 $pdo->exec("
17 CREATE TABLE IF NOT EXISTS jugadores (
18     id INTEGER PRIMARY KEY AUTOINCREMENT,
19     usuario TEXT NOT NULL UNIQUE,
20     nombre TEXT NOT NULL,
21     apellidos TEXT NOT NULL,
22     created_at TEXT DEFAULT (datetime('now'))
23 );
24 ");
25
26 // Seed de 5 jugadores por defecto si la tabla está vacía
27 $count = (int)$pdo->query("SELECT COUNT(*) FROM jugadores")->fetchColumn();
28 if ($count === 0) {
29     $pdo->beginTransaction();
30     $stmt = $pdo->prepare("
31         INSERT INTO jugadores (usuario, nombre, apellidos)
32         VALUES (:usuario, :nombre, :apellidos)
33     ");
34
35     $seed = [
36         ['u' => 'fperez', 'n' => 'Francisco', 'a' => 'Pérez Sánchez'],
37         ['u' => 'mlopez', 'n' => 'María', 'a' => 'López Díaz'],
38         ['u' => 'aperez', 'n' => 'Alberto', 'a' => 'Pérez Martín'],
39         ['u' => 'cserrano', 'n' => 'Carmen', 'a' => 'Serrano Ruiz'],
40         ['u' => 'rnavarro', 'n' => 'Raúl', 'a' => 'Navarro Gil'],
41     ];
42
43     foreach ($seed as $s) {
44         $stmt->execute([
45             ':usuario' => $s['u'],
46             ':nombre' => $s['n'],
47             ':apellidos' => $s['a'],
48         ]);
49     }
50     $pdo->commit();
51 }
```

Modelo Jugador.php

```

1  <?php
2  declare(strict_types=1);
3
4  class Jugador {
5      private PDO $pdo;
6
7      public function __construct(PDO $pdo) {
8          $this->pdo = $pdo;
9      }
10
11     public function all(): array {
12         $stmt = $this->pdo->query("SELECT id, usuario, nombre, apellidos, created_at FROM jugadores ORDER BY id DESC");
13         return $stmt->fetchAll(PDO::FETCH_ASSOC);
14     }
15
16     public function find(int $id): ?array {
17         $stmt = $this->pdo->prepare("SELECT id, usuario, nombre, apellidos FROM jugadores WHERE id = :id");
18         $stmt->execute([':id' => $id]);
19         $row = $stmt->fetch(PDO::FETCH_ASSOC);
20         return $row ?: null;
21     }
22
23     public function create(array $data): array {
24         $errors = $this->validate($data);
25         if ($errors) { return [false, $errors]; }
26
27         try {
28             $stmt = $this->pdo->prepare("
29             INSERT INTO jugadores(usuario, nombre, apellidos)
30             VALUES(:usuario, :nombre, :apellidos)
31             ");
32             $stmt->execute([
33                 ':usuario' => $data['usuario'],
34                 ':nombre' => $data['nombre'],
35                 ':apellidos' => $data['apellidos']
36             ]);
37             return [true, []];
38         } catch (PDOException $e) {
39             if (str_contains($e->getMessage(), 'UNIQUE')) {
40                 return [false, ['usuario' => 'El usuario ya existe']];
41             }
42             throw $e;
43         }
44     }
45
46     public function update(int $id, array $data): array {
47         $errors = $this->validate($data, $id);
48         if ($errors) { return [false, $errors]; }
49
50         try {
51             $stmt = $this->pdo->prepare("
52             UPDATE jugadores
53             SET usuario = :usuario, nombre = :nombre, apellidos = :apellidos
54             WHERE id = :id
55             ");
56             $stmt->execute([
57                 ':usuario' => $data['usuario'],
58                 ':nombre' => $data['nombre'],
59                 ':apellidos' => $data['apellidos'],
60                 ':id' => $id
61             ]);
62             return [true, []];
63         } catch (PDOException $e) {
64             if (str_contains($e->getMessage(), 'UNIQUE')) {
65                 return [false, ['usuario' => 'El usuario ya existe']];
66             }
67             throw $e;
68         }
69     }
70

```

```
71     public function delete(int $id): void {
72         $stmt = $this->pdo->prepare("DELETE FROM jugadores WHERE id = :id");
73         $stmt->execute([':id' => $id]);
74     }
75
76     private function validate(array $data, ?int $id = null): array {
77         $errors = [];
78         if ($data['usuario'] === '') { $errors['usuario'] = 'Usuario obligatorio'; }
79         if ($data['nombre'] === '') { $errors['nombre'] = 'Nombre obligatorio'; }
80         if ($data['apellidos'] === '') { $errors['apellidos'] = 'Apellidos obligatorios'; }
```

Controlador **JugadorController.php**


```

1  <?php
2
3  declare(strict_types=1);
4
5  require_once __DIR__ . '/../models/Jugador.php';
6
7  class JugadorController
8  {
9      private PDO $pdo;
10     private Jugador $model;
11
12     public function __construct(PDO $pdo)
13     {
14         $this->pdo = $pdo;
15         $this->model = new Jugador($pdo);
16     }
17
18     public function index(): void
19     {
20         $jugadores = $this->model->all();
21         $this->render('jugadores/index', compact('jugadores'));
22     }
23
24     public function crear(): void
25     {
26         $jugador = ['usuario' => '', 'nombre' => '', 'apellidos' => ''];
27         $errors = [];
28         $this->render('jugadores/crear', compact('jugador', 'errors'));
29     }
30
31     public function store(): void
32     {
33         $data = [
34             'usuario' => trim($_POST['usuario'] ?? ''),
35             'nombre' => trim($_POST['nombre'] ?? ''),
36             'apellidos' => trim($_POST['apellidos'] ?? '')
37         ];
38         [$ok, $errors] = $this->model->create($data);
39
40         if ($ok) {
41             header('Location: ?action=index');
42             exit;
43         }
44         $jugador = $data;
45         $this->render('jugadores/crear', compact('jugador', 'errors'));
46     }
47
48     public function editar(?int $id): void
49     {
50         if (!$id) {
51             $this->badRequest('Falta id');
52             return;
53         }
54     }
55 }

```

```

53     }
54     $jugador = $this->model->find($id);
55     if (!$jugador) {
56         $this->notFound();
57         return;
58     }
59     $errors = [];
60     $this->render('jugadores/editar', compact('jugador', 'errors'));
61 }
62
63 public function update(?int $id): void
64 {
65     if (!$id) {
66         $this->badRequest('Falta id');
67         return;
68     }
69     $data = [
70         'usuario' => trim($_POST['usuario'] ?? ''),
71         'nombre' => trim($_POST['nombre'] ?? ''),
72         'apellidos' => trim($_POST['apellidos'] ?? '')
73     ];
74     [$ok, $errors] = $this->model->update($id, $data);
75     if ($ok) {
76         header('Location: ?action=index');
77         exit;
78     }
79     $jugador = array_merge(['id' => $id], $data);
80     $this->render('jugadores/editar', compact('jugador', 'errors'));

```

Capturas del proyecto en funcionamiento:

Listado de jugadores

[Nuevo jugador](#)

ID	Usuario	Nombre	Apellidos	Acciones
4	cserrano	Carmen	Serrano Ruiz	Editar Eliminar
3	aperez	Alberto	Pérez Martín	Editar Eliminar
2	mlopez	María	López Díaz	Editar Eliminar
1	jgarcia	Juan	García López	Editar Eliminar

Nuevo jugador

Usuario

Nombre

Apellidos

Crear

Volver

Editar jugador #4

Usuario

cserrano

Nombre

Carmen

Apellidos

Serrano Ruiz

Actualizar

Volver

P7 - Manejo de ficheros

Aquí se trabaja con archivos directamente desde PHP: lectura, escritura, CSV y PDF.

Ejemplo 1 - Crear/escribir y leer un archivo de texto

Crea (o sobrescribe) `notas.txt`, escribe dos líneas y luego lee el archivo línea a línea.

ejemplo1.php · líneas 1-16

```
1 <?php
2 $archivo = 'notas.txt';
3
4 // 1. Escribir
5 $texto = "Primera línea de texto\nSegunda línea\n";
6 $f = fopen($archivo, 'w');
7 fwrite($f, $texto);
8 fclose($f);
9
10 // 2. Leer
11 $f = fopen($archivo, 'r');
12 while (!feof($f)) {
13     echo fgets($f) . "<br>";
14 }
15 fclose($f);
16 ?>
```

Ejemplo 2 - Leer un archivo completo

Lee el contenido entero con `file_get_contents` y también lo divide en un array con `file()`.

ejemplo2.php · líneas 1-5

```
1 <?php
2 echo nl2br(file_get_contents('notas.txt')); // Lee todo el contenido
3 $lineas = file('notas.txt'); // devuelve array con cada línea
4 print_r($lineas);
5 ?>
```

Ejemplo 3 - Añadir al final (modo 'a')

Modo "append": añade contenido al final del archivo sin borrar lo anterior.

ejemplo3.php · líneas 1-6

```
1 <?php
2 $archivo = 'notas.txt';
3 $f = fopen($archivo, 'a');
4 fwrite($f, "Nueva línea añadida.\n");
5 fclose($f);
6 ?>
```

Ejemplo 4 - Generar y descargar un CSV

Crea un archivo CSV con `fputcsv` y manda cabeceras para que el navegador lo descargue.

ejemplo4.php · líneas 1-17

```
1 <?php
2 $datos = [
3     ['Nombre', 'Edad', 'Ciudad'],
4     ['Francisco', 21, 'Sevilla'],
5     ['Lucía', 25, 'Málaga']
6 ];
7
8 $fp = fopen('personas.csv', 'w');
9 foreach ($datos as $fila) {
10     fputcsv($fp, $fila, ';');
11 }
12 fclose($fp);
13
14 header('Content-Type: text/csv');
15 header('Content-Disposition: attachment; filename="personas.csv"');
16 readfile('personas.csv');
17 ?>
```

Ejemplo 5 - PDF con FPDF

Primer PDF con la librería **FPDF**: añadir página, fuente y un par de líneas de texto.

ejemplo5.php · líneas 1-13

```
1 <?php
2 require_once 'fpdf186/fpdf.php';
3
4 $pdf = new FPDF();
5 $pdf->AddPage();
6 $pdf->SetFont('Arial', 'B', 16);
7 $pdf->Cell(0, 10, 'Ejemplo de exportacion PDF', 0, 1, 'C');
8
9 $pdf->SetFont('Arial', '', 12);
10 $pdf->Cell(0, 10, 'Generado con PHP + FPDF', 0, 1, 'L');
11
12 $pdf->Output('I', 'demo.pdf'); // I: Ver en navegador, D: Descargar
13 ?>
```

Ejemplo 6 - Convertir `notas.txt` a PDF

Lee el archivo de texto línea a línea y vuelca cada línea en un PDF.

ejemplo6.php · líneas 1-16

```
1 <?php
2 require_once 'fpdf186/fpdf.php';
3
4 $contenido = file('notas.txt');
5 $pdf = new FPDF();
6 $pdf->AddPage();
7 $pdf->SetFont('Arial', '', 12);
8
9 foreach ($contenido as $linea) {
10     //$pdf->Cell(0, 10, utf8_decode(trim($linea)), 0, 1);
11     $pdf->Cell(0, 10, mb_convert_encoding(trim($linea), 'ISO-8859-1', 'UTF-8'), 0, 1);
12     //$pdf->Cell(0, 10, iconv('UTF-8', 'ISO-8859-1//IGNORE', trim($linea)), 0, 1);
13 }
14
15 $pdf->Output('I', 'notas.pdf');
16 ?>
```

Capturas del manejo de ficheros en navegador:

Primera línea de texto

Segunda línea

Primera línea de texto

Segunda línea

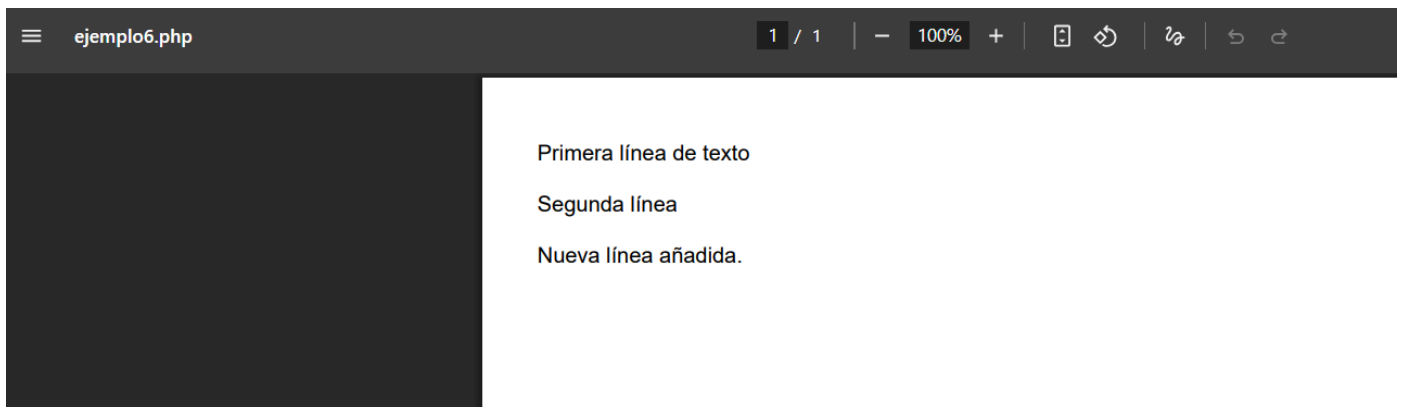
Array ([0] => Primera línea de texto [1] => Segunda línea)

DWES_FranciscoPerez > Entrega5 > P7_Files > ≡ notas.txt

```
1 Primera línea de texto
2 Segunda línea
3 Nueva línea añadida.
4
```

DWES_FranciscoPerez > Entrega5 > P7_Files > personas.csv

	A	B	C	D	
1	Nomb...	Edad	Ciudad		
2	Ana	22	Sevilla		
3	Juan	25	Málaga		
4					



P8 - MySQLi + PDF

El apartado final junta dos cosas que ya se han visto por separado: **MySQLi** para leer datos + **FPDF** para generar el documento.

El script se conecta a **dwes**, lee la tabla **ud4_coches** (id, marca, modelo, año) y genera un PDF con esos datos en formato de tabla.


```
1 <?php
2 // Importar la biblioteca FPDF
3 require_once 'fpdf186/fpdf.php';
4
5 // Conexión a la base de datos
6 $mysqli = new mysqli("localhost", "root", "", "dwes");
7
8 // Verificar conexión
9 if ($mysqli->connect_error) {
10     die("Error en la conexión: " . $mysqli->connect_error);
11 }
12
13 // Consultar los datos de la tabla 'coches'
14 $query = "SELECT id, marca, modelo, anio FROM ud4_coches";
15 $resultado = $mysqli->query($query);
16
17 // Crear un nuevo PDF
18 $pdf = new FPDF();
19 $pdf->AddPage();
20 $pdf->SetFont('Arial', 'B', 12);
21
22 // Título del documento
23 $pdf->Cell(0, 10, 'Lista de Coches', 0, 1, 'C');
24 $pdf->Ln(10); // Salto de línea
25
26 // Encabezados de la tabla
27 $pdf->Cell(20, 10, 'ID', 1, 0, 'C');
28 $pdf->Cell(50, 10, 'Marca', 1, 0, 'C');
29 $pdf->Cell(50, 10, 'Modelo', 1, 0, 'C');
30 $pdf->Cell(30, 10, 'Anio', 1, 1, 'C');
31
32 // Agregar los datos a la tabla
33 while ($fila = $resultado->fetch_assoc()) {
34     $pdf->Cell(20, 10, $fila['id'], 1, 0, 'C');
35     $pdf->Cell(50, 10, $fila['marca'], 1, 0, 'C');
36     $pdf->Cell(50, 10, $fila['modelo'], 1, 0, 'C');
37     $pdf->Cell(30, 10, $fila['anio'], 1, 1, 'C');
38 }
39
40 // Cerrar la conexión
41 $mysqli->close();
42
43 // Salida del PDF
44 $pdf->Output('D', 'coches.pdf');
45 ?>
```

Captura del PDF generado:

coches.pdf 1 / 1 100%

ID	Marca	Modelo	Año
1	Toyota	Corolla	2020
2	Honda	Civic	2018
3	Ford	Focus	2019

Conclusiones

- **MySQLi** y **PDO** son dos formas válidas de hablar con MySQL. PDO es más portable y suele ser la opción recomendada hoy en día.
- Las **transacciones** son fundamentales cuando varias operaciones tienen que ir "todo o nada". Si una falla, se hace **rollback** y la base de datos queda como estaba.
- La **inyección SQL** se evita SIEMPRE con prepared statements. Concatenar entrada del usuario en SQL es uno de los errores más graves que se pueden cometer.
- El **patrón MVC** ordena el código en proyectos medianos/grandes. Separa lógica, presentación y datos.
- **SQLite** es perfecto para proyectos pequeños: BD en un archivo, sin servidor.
- **FPDF** es la forma más clásica de generar PDFs en PHP. Para informes a partir de la BD, es lo más rápido.

Esta es la unidad donde por fin se siente que PHP es "un lenguaje serio para hacer aplicaciones reales", porque ya hay base de datos, capas separadas y exportación a formatos profesionales.